

CYBER FLORIDA AI VOICE ASSISTANT

PIXEL

Project Description, Requirements, Architecture & End-to-End Development Guide

PROJECT PURPOSE

Design and build Pixel as a secure, voice-first conversational assistant for Cyber Florida at USF. Pixel should help users discover Cyber Florida information and resources, understand cybersecurity topics, receive defensive security guidance, and complete approved actions through natural conversation.

Prepared as a development blueprint for AI agents, engineers, designers, security reviewers, and project stakeholders.

Document version: 1.0 | August 2026

Pixel should feel like a specialized Cyber Florida digital assistant - not a generic chatbot with a microphone.

Executive Summary

Pixel is an AI-powered, voice-first assistant designed for Cyber Florida at the University of South Florida. The product goal is to give students, educators, public-sector personnel, cybersecurity professionals, businesses, and general visitors a natural conversational way to access Cyber Florida information and obtain trustworthy cybersecurity guidance.

Cyber Florida at USF, formally The Florida Center for Cybersecurity, was created within the University of South Florida and works to position Florida as a national leader in cybersecurity through education, research, and outreach. Pixel should extend that mission by making approved knowledge and digital services easier to discover and use through voice and text interaction.

The recommended implementation is a modular system that separates the user interface, real-time voice transport, speech processing, AI orchestration, Cyber Florida knowledge retrieval, approved actions, security controls, and observability. This prevents Pixel from becoming tightly coupled to a single AI model or speech vendor and makes the platform easier to improve over time.

MVP DEFINITION

The first production version should reliably support: push-to-talk or hands-free listening, speech-to-text, short-term conversational context, Cyber Florida RAG search, cybersecurity Q&A, scam/phishing guidance, natural text-to-speech, source-aware answers, text fallback, interruption while Pixel is speaking, basic analytics, safety controls, and a production administration workflow for updating the knowledge base.

What this document provides

- A precise project description and product vision for Pixel.
- Functional, non-functional, technical, security, privacy, data, UX, and operational requirements.
- A recommended system architecture and component responsibilities.
- A complete development method from product discovery through production deployment.
- Step-by-step implementation tasks, test strategy, release gates, risk controls, and definition of done.
- A practical repository structure and API/tool boundaries that an AI coding agent can follow.

Document Map

SECTION	WHAT IT COVERS
1. Project Description	Purpose, vision, users, use cases, assistant identity, scope and success criteria.
2. Project Requirements	Functional, non-functional, technical, knowledge, security, privacy, UX and operations requirements.
3. Recommended Architecture	Voice pipeline, AI orchestration, RAG, tools, data stores, trust boundaries and request flow.
4. Development Workflow	Fourteen phases from initial discovery to production operations.
5. Step-by-Step Build Plan	Concrete engineering sequence and implementation checklist.
6. Testing & Evaluation	Unit, integration, voice, RAG, safety, performance, accessibility and user acceptance testing.
7. Deployment & Operations	Environments, CI/CD, secrets, monitoring, incident response and content maintenance.
8. Risks & Controls	Primary technical, product, security and adoption risks with mitigations.
9. Definition of Done	What must be true before Pixel is considered production-ready.
10. Source Notes	Authoritative Cyber Florida context and recommended standards.

1. Project Description

1.1 Product statement

Pixel is a secure, conversational AI voice assistant for Cyber Florida. It should allow a person to speak naturally, receive a fast spoken response, continue the conversation without repeating context, and be guided toward the correct Cyber Florida resource or defensive cybersecurity action.

The experience is inspired by mainstream voice assistants such as Siri and Alexa, but Pixel is not intended to copy their broad consumer scope. Its advantage is specialization: Cyber Florida knowledge, cybersecurity education, safer digital behavior, program discovery, and approved Cyber Florida actions.

ONE-SENTENCE PRODUCT DEFINITION

Pixel is Cyber Florida's voice and conversational interface: a trustworthy AI assistant that listens, understands, retrieves approved information, reasons within defined boundaries, speaks naturally, and guides users to the correct next step.

1.2 Product vision

The long-term vision is for Pixel to become an intelligent digital front door for Cyber Florida. Instead of forcing users to search multiple webpages, program pages, FAQs, documents, and resource collections, Pixel should identify the user's intent and provide the right explanation, link, resource, or approved action in a conversational format.

- Voice-first, but never voice-only: every important interaction should also be available through text and accessible UI controls.
- Source-grounded: organization-specific claims should be generated from approved Cyber Florida knowledge whenever possible.
- Security-first: the assistant must behave consistently with the expectations of a cybersecurity organization.
- Action-oriented: Pixel should help the user reach a next step instead of merely producing information.
- Modular: speech, model, retrieval, action and analytics components should be replaceable without rewriting the entire platform.

1.3 Target users

USER GROUP	PRIMARY VALUE
General public	Cybersecurity awareness, scam/phishing guidance, account safety, Cyber Florida information.
Students	Programs, training, career pathways, internships, events, cybersecurity concepts.
Educators	K-12/college resources, program discovery, cybersecurity education support.
Veterans / first responders / public servants	Workforce programs, cyber training, career transition resources.
Cybersecurity professionals	Advanced resources, events, research, professional development information.
Public-sector organizations	Education, exercises, cyber resilience and available Cyber Florida programs.
Businesses / organizations	Cybersecurity awareness, resources, defensive guidance and appropriate referrals.
Cyber Florida staff	A consistent digital interface for frequently requested public information and approved workflows.

1.4 Primary use cases

- "What is Cyber Florida and what does it do?"
- "Which Cyber Florida program is relevant to me?"
- "What training is available for someone starting a cybersecurity career?"
- "Explain phishing in simple terms."
- "I clicked a suspicious link. What should I do now?"
- "Does this message show common scam warning signs?"
- "Show me the page for this program."
- "When is this event?" - answered only after retrieving current authoritative information.
- "Continue explaining the program you just mentioned." - handled through conversational context.

1.5 Pixel identity and behavior

ATTRIBUTE	REQUIREMENT
Name	Pixel
Role	Cyber Florida AI voice and conversational assistant
Default tone	Professional, clear, calm, concise, security-conscious
Answer length	Usually 1-4 spoken sentences; expand when the user asks for detail
Disclosure	Pixel must identify itself as an AI assistant when relevant and never imply it is a human employee.
Knowledge behavior	Use approved Cyber Florida sources for organization-specific information; distinguish verified information from general guidance.
Uncertainty behavior	State when information cannot be verified instead of inventing details.
Safety behavior	Prioritize defensive, educational and authorized cybersecurity assistance.

1.6 Scope: MVP vs. later releases

SCOPE	CONTENT
MVP	Voice input/output; text fallback; multi-turn context; Cyber Florida RAG; cybersecurity Q&A; phishing/scam guidance; source links; navigation actions; user interruption; feedback; observability; knowledge admin workflow; safety/security controls.
Release 2	Personalized program recommendations; richer website actions; multimodal screenshot/document analysis; user profiles where appropriate; more advanced event/opportunity discovery.
Release 3+	Mobile-native experiences; deeper authenticated integrations; proactive reminders/notifications where approved; multilingual support; richer organization workflows; hardware/kiosk modes.
Out of scope for MVP	General-purpose smart-home control, unrestricted device automation, autonomous high-impact security actions, unrestricted access to internal Cyber Florida systems, or long-term memory of sensitive user information.

1.7 Success criteria

- Users can start a conversation with minimal friction and reliably complete a full listen-understand-answer cycle.
- Pixel answers organization-specific questions from approved sources and can show or provide the source used.

- Users can interrupt Pixel during speech and continue naturally.
- Common cybersecurity education questions are answered clearly at the user's level.
- Potentially high-risk or uncertain questions are handled safely and transparently.
- The knowledge base can be updated without redeploying the complete application.
- The engineering team can measure latency, retrieval quality, failures, user feedback, and safety events.

04

2. Project Requirements

Requirements should be tracked with identifiers so they can be linked to design decisions, implementation tickets, tests, and release gates.

2.1 Functional requirements

ID	CAPABILITY	REQUIREMENT
FR-01	Voice capture	Capture microphone audio after explicit user initiation or approved wake/listening mode.
FR-02	Voice activity detection	Detect speech start/end to reduce unnecessary audio transmission and improve turn-taking.
FR-03	Speech recognition	Convert spoken language to text with enough accuracy for cybersecurity terminology and common proper nouns.
FR-04	Text input	Provide keyboard/text input as a complete fallback to voice.
FR-05	Streaming response	Begin producing the response before the full answer is generated when the selected architecture supports streaming.
FR-06	Text-to-speech	Speak Pixel responses with natural pacing and clear pronunciation.
FR-07	Barge-in	Allow the user to interrupt Pixel while it is speaking; stop playback and process the new request.
FR-08	Conversation state	Retain recent-turn context so follow-up questions resolve references such as "that program" or "what about eligibility?".
FR-09	Intent routing	Classify requests into Cyber Florida knowledge, general cybersecurity guidance, scam/phishing help, navigation/action, clarification, or unsupported intent.
FR-10	Cyber Florida RAG	Retrieve relevant approved Cyber Florida content before answering organization-specific questions.
FR-11	Source attribution	Associate retrieved answers with the underlying source URL/document and expose sources in the UI when useful.
FR-12	Freshness handling	Treat dates, deadlines, events, leadership, availability, and other time-sensitive information as data that must be refreshed or verified.
FR-13	Program discovery	Ask concise questions and recommend relevant Cyber Florida programs/resources using verified eligibility/content data.
FR-14	Cybersecurity education	Explain defensive cybersecurity concepts at beginner, intermediate, or advanced depth.
FR-15	Incident guidance	Provide prioritized defensive steps for common user incidents such as phishing clicks, credential exposure, account compromise, and suspicious messages.
FR-16	Scam analysis	Identify warning signs in user-provided message content without claiming certainty

ID	CAPABILITY	REQUIREMENT
		beyond available evidence.
FR-17	Approved navigation	Open or navigate to approved Cyber Florida pages from the website/app.
FR-18	Tool confirmation	For any action with meaningful side effects, present a confirmation boundary before execution unless the action is inherently safe and reversible.
FR-19	Error recovery	Handle microphone, network, retrieval, model, speech synthesis and tool errors with clear user-facing recovery options.
FR-20	Feedback	Allow users to rate or flag answers and optionally provide a reason.
FR-21	Admin ingestion	Authorized staff can add/update/remove approved knowledge sources and trigger re-indexing.
FR-22	Content lifecycle	Track source, version/fetch time, last updated time and document status for indexed knowledge.
FR-23	Analytics	Measure turns, completion, latency, retrieval success, errors and feedback without storing unnecessary sensitive content.
FR-24	Session controls	Allow the user to clear the current conversation/session context.
FR-25	Accessibility	Provide captions/transcripts, keyboard navigation, focus states and non-audio alternatives.

2.2 Non-functional requirements

ID	AREA	REQUIREMENT
NFR-01	Latency	Optimize for conversational responsiveness. Instrument time-to-transcript, retrieval, model first token/audio, and total turn duration independently.
NFR-02	Availability	The public experience should degrade gracefully when a dependency is unavailable; text mode and basic navigation should remain usable where possible.
NFR-03	Scalability	Stateless application services should scale horizontally; session state and caches should live in appropriate shared stores.
NFR-04	Security	Apply least privilege, encrypted transport, secret management, input validation, dependency scanning, logging controls and separation of public vs. privileged actions.
NFR-05	Privacy	Minimize audio/text retention, avoid sensitive data in logs, document retention choices, and provide clear user notice for recording/processing where applicable.
NFR-06	Accessibility	Target WCAG 2.2 AA for the web interface and ensure voice functions have equivalent visual/text controls.
NFR-07	Maintainability	Keep providers behind interfaces; isolate speech, LLM, retrieval, tools and storage concerns; document boundaries and contracts.
NFR-08	Observability	Use structured logs, metrics, traces, correlation IDs and dashboards for the complete voice/AI request path.

ID	AREA	REQUIREMENT
NFR-09	Testability	Core orchestration and tool logic must be testable without a microphone or live AI provider by using mocks/fixtures.
NFR-10	Auditability	Record privileged tool calls and administrative content changes with actor, timestamp, target and result.
NFR-11	Resilience	Use timeouts, retries with limits, circuit-breaker behavior where appropriate, and explicit fallbacks for dependent services.
NFR-12	Portability	Avoid hard-coding a single vendor into core domain logic; use provider adapters and configuration.

2.3 Technical requirements

COMPONENT	RECOMMENDED REQUIREMENT
Client	Modern responsive web client first; TypeScript/React-based implementation is a strong default. Use mobile-native code later only if platform-specific capabilities justify it.
Real-time transport	Use WebRTC where low-latency bi-directional audio streaming is needed; WebSocket is acceptable for simpler streaming/control channels.
Backend	A Python FastAPI service is a strong default for orchestration/RAG because of the AI ecosystem, typed APIs and async support. Node/TypeScript is also valid if the team standardizes there.
AI provider layer	Define interfaces for LLM/realtime model, speech-to-text and text-to-speech. Business logic should consume the interfaces, not vendor-specific SDK objects.
Knowledge retrieval	Start with PostgreSQL + pgvector for embeddings/vector search, metadata filtering and operational simplicity. Add a dedicated vector engine only if scale/latency measurements require it.
Content storage	Store raw approved documents/pages in durable object storage or a managed document store; keep indexed chunks traceable back to a source/version.
Session state	Use secure server-side session identifiers; Redis or equivalent can hold short-lived conversation/session state when needed.
Authentication	Public anonymous mode for general information; OIDC/SAML or organization-approved SSO for protected staff/admin features.
Deployment	Containerize application services. Use separate development, staging and production environments with managed secrets and CI/CD promotion.
Observability	Centralized logs, traces, metrics and alerting. Every user turn should have a correlation ID across client gateway, retrieval, model and tools.

2.4 Knowledge and data requirements

- Maintain an allowlist of approved domains/documents that Pixel may treat as authoritative for Cyber Florida facts.
- Ingest pages/documents with metadata: title, canonical URL, content type, program/topic, audience, publication/update date, fetch timestamp and status.
- Normalize and chunk content semantically; preserve headings and source boundaries so retrieved passages remain interpretable.
- Create embeddings and store them with metadata filters and deterministic source identifiers.
- Run scheduled or manual refresh jobs and detect changed/deleted content.

- Do not let text retrieved from a page override system/developer instructions; retrieved content is untrusted data.
- Separate public knowledge from internal/privileged knowledge with explicit access controls and separate indexes where needed.
- Maintain a small curated facts layer for critical information that requires deterministic answers, such as official contact points or approved emergency guidance.

2.5 Security and privacy requirements

AREA	REQUIREMENT
Prompt injection	Treat web/document text as untrusted. Strip or ignore embedded instructions; tools are available only through server-side allowlists and policy checks.
Secrets	Never expose API keys or service credentials to the browser. Store secrets in a managed secrets service or protected environment configuration.
Authorization	Enforce authorization on the server for every protected action. UI visibility is not an authorization control.
Sensitive data	Detect/redact high-risk values from application logs where feasible. Never intentionally log passwords, one-time codes or authentication tokens.
Audio	Define whether raw audio is transient or retained. Prefer transient processing by default and document exceptions.
Abuse/rate limits	Apply rate limits, request size limits and abuse controls to public endpoints.
Tool safety	High-impact or side-effecting tools require explicit schemas, validation, authorization and confirmation policies.
Dependencies	Perform software composition/dependency scanning and patch critical vulnerabilities through a defined process.
Admin changes	Require authentication and audit logs for knowledge source changes, policy updates and production configuration.
Incident response	Document how to disable a tool/provider, rotate secrets, remove a bad source, roll back a deployment and investigate suspicious activity.

2.6 UX and voice requirements

- Clearly display listening, processing, speaking and error states.
- Never make the microphone state ambiguous. The user must be able to stop listening immediately.
- Show a live transcript or post-turn transcript where appropriate.
- Provide visible Stop/Mute controls while Pixel is speaking.
- Keep voice answers short; use the screen for links, sources, multi-step instructions and detailed content.
- Design for interruption and fast turn-taking rather than treating each answer as a monologue.
- Use explicit wording when Pixel is uncertain or cannot verify a fact.
- Make source links and recommended next actions prominent after a relevant answer.
- All controls must be operable by keyboard and have accessible labels.

2.7 Minimum acceptance criteria for MVP

CATEGORY	MVP ACCEPTANCE CRITERION
Conversation	A user can speak a question, receive a spoken answer, ask a follow-up and interrupt the assistant.
Knowledge	A test set of approved Cyber Florida questions meets the project's defined grounded-answer quality target and surfaces correct sources.
Safety	Pixel does not expose secrets/system instructions in standard

CATEGORY	MVP ACCEPTANCE CRITERION
	red-team tests and does not execute unauthorized tool actions.
Reliability	Voice, text, retrieval and error fallbacks work in staging under expected load.
Accessibility	Core flows pass keyboard testing and automated/manual accessibility checks.
Operations	Teams can monitor errors/latency, update knowledge, review feedback and roll back a release.

05

3. Recommended System Architecture

3.1 Architecture principles

- Separate conversation UX from AI provider implementation.
- Keep privileged tools server-side.
- Use retrieval for Cyber Florida facts instead of relying on model memory.
- Treat voice as a real-time transport problem with explicit state management.
- Make every turn observable through correlation IDs and component-level timings.
- Design fail-safe behavior before adding advanced features.

3.2 Logical architecture

LAYER	RESPONSIBILITIES
1. Client layer	Web UI, microphone permissions, audio capture/playback, transcript, assistant state animation, text fallback, source cards, actions.
2. Real-time gateway	Session negotiation, WebRTC/WebSocket transport, authentication context, rate limits, audio/control events.
3. Speech layer	Voice activity detection, speech-to-text, pronunciation handling, text-to-speech, streaming audio.
4. AI orchestrator	System policy, conversation state, intent routing, tool selection, retrieval decision, answer generation, safety checks.
5. Knowledge layer	Crawler/importers, content normalization, chunking, embeddings, vector search, reranking, metadata filters, source links.
6. Action/tool layer	Website navigation payloads, program/event lookup, approved search, future authenticated actions.
7. Data layer	PostgreSQL/pgvector, object storage, short-lived session store, configuration and audit records.
8. Security/observability	Authentication/authorization, policy enforcement, secret management, structured logging, metrics, traces, alerting.

REFERENCE FLOW

User speaks -> Client captures audio -> Real-time gateway -> STT / realtime speech processing -> Orchestrator identifies intent -> RAG and/or approved tool executes -> Orchestrator composes grounded answer -> Safety/output checks -> TTS -> audio + transcript + sources returned to user.

3.3 Request flow: Cyber Florida knowledge question

1. User asks a question by voice or text.
2. Client creates a turn ID and sends the input to the backend/realtime session.

3. Speech input is transcribed and normalized.
4. Intent router identifies the request as organization-specific knowledge.
5. Orchestrator creates a retrieval query using the current question plus necessary conversation context.
6. Retriever searches approved indexed Cyber Florida content and returns top candidate passages with metadata.
7. Optional reranking filters to the most relevant evidence.
8. Model generates an answer constrained by retrieved evidence and response-style rules.
9. Output validator checks for unsupported facts, unsafe content, invalid tool results and formatting issues.
10. TTS generates spoken audio; client displays transcript and source cards.
11. Telemetry records timing, retrieval IDs, success/error state and non-sensitive quality signals.

3.4 Request flow: defensive cybersecurity guidance

1. Classify whether the request is educational, incident-response guidance, suspicious-message analysis, or a potentially harmful cyber request.
2. For normal defensive questions, answer directly using approved safety guidance and current knowledge sources where required.
3. For incident scenarios, prioritize containment and account protection steps before explanation.
4. Ask for clarification only when it changes the recommended action materially.
5. Avoid collecting secrets, passwords, one-time codes or sensitive identifiers.
6. Escalate to an official support/security/law-enforcement path when the situation requires authority or investigation Pixel does not have.

3.5 Suggested backend service boundaries

SERVICE	RESPONSIBILITY
api-gateway	REST endpoints, session bootstrap, auth context, health checks, rate limiting.
realtime-service	Voice session transport, turn lifecycle, interruption events, streaming control.
orchestrator	Conversation state, intent routing, AI calls, retrieval/tool policies, answer assembly.
knowledge-service	Search, reranking, source metadata, ingestion status.
ingestion-worker	Fetch/import, clean, chunk, embed, index, update/delete handling.
tool-service	Server-side approved actions with schema validation and authorization.
analytics/telemetry	Metrics/events pipeline and quality dashboards. Can begin embedded in backend before separation is justified.

3.6 Recommended repository structure

```

pixel/
  apps/
    web/          # React/Next.js client
    api/          # FastAPI public/backend API
    worker/       # ingestion/background jobs
  packages/
    ai/           # model provider adapters + prompts/policies
    voice/        # STT/TTS/realtime interfaces
    knowledge/    # retrieval + chunking + citation models
    tools/        # approved tool schemas and implementations
    security/     # authz, redaction, policy checks
    observability/ # logging/tracing/metrics helpers

```

```

shared/          # shared types/config
evals/
  knowledge/     # grounded QA evaluation set
  safety/        # abuse/prompt injection tests
  voice/         # transcript/latency fixtures
infra/           # containers/IaC/deploy configuration
docs/            # ADRs, API contracts, runbooks
scripts/         # local setup and admin utilities

```

3.7 Core API/event contracts

ENDPOINT / EVENT	PURPOSE
POST /v1/sessions	Create a conversation session and return permitted capabilities/configuration.
POST /v1/messages	Text-mode user message; returns/streams assistant response and sources.
POST /v1/realtime/token or equivalent	Issue short-lived session material for real-time voice transport without exposing long-lived server credentials.
GET /v1/sources/{id}	Return approved source metadata if the UI needs a normalized source view.
POST /v1/feedback	Record rating/issue category for a turn.
POST /admin/sources	Authorized source registration/import.
POST /admin/reindex	Authorized knowledge refresh/re-index operation.
GET /admin/ingestion/{job}	Authorized ingestion status and errors.

06

4. Development Workflow: From Scratch to Production

The safest development method is to build Pixel in vertical slices. Prove one complete conversational path early, then strengthen knowledge, actions, security, evaluation and operations. Avoid building a large UI or large RAG pipeline before the end-to-end voice loop works.

Phase 0 - Product Discovery and Constraints

Objective: Turn the idea into a precise product problem, scope and measurable MVP.

Work to complete

- Identify project owner, technical owner, security reviewer, content owner and approver roles.
- List target users and top 20 user questions/tasks.
- Decide public vs. authenticated capabilities.
- Define what Pixel must never do and which topics require escalation.
- Inventory Cyber Florida content sources and classify public vs. internal.
- Define data retention expectations for audio, transcripts, feedback and logs.
- Document MVP and deferred features.

Deliverables

- Product brief
- MVP scope
- Initial use-case list
- Data/content inventory
- Risk assumptions
- Stakeholder map

Exit criteria

- Stakeholders agree on the MVP
- Content owners identify authoritative sources
- Security/privacy constraints are recorded before implementation

Phase 1 - Assistant Identity, Conversation Policy and Safety Rules

Objective: Specify exactly how Pixel behaves before model prompts are written.

Work to complete

- Write identity: name, role, disclosure language and relationship to Cyber Florida.
- Define response style for voice vs. screen.
- Define uncertainty wording and source-grounding rules.
- Define defensive cybersecurity assistance boundaries.
- Define when to ask follow-up questions and when to escalate.
- Define tool/action confirmation rules.
- Create 50-100 representative conversations including failure cases.

Deliverables

- System behavior specification
- Conversation examples
- Safety rules
- Escalation matrix
- Tool confirmation policy

Exit criteria

- Reviewers can predict how Pixel should respond to common and difficult scenarios
- Examples cover both good-path and failure-path behavior

Phase 2 - Repository, Environments and Engineering Foundation

Objective: Create a reproducible development base with strong boundaries and automated checks.

Work to complete

- Create monorepo or clearly separated frontend/backend repositories.
- Configure TypeScript frontend and Python backend with formatters, linters and tests.
- Create .env.example without real secrets.
- Add container development files and local start commands.
- Create development, staging and production configuration patterns.
- Add CI for lint, type-check, unit tests and dependency/security checks.
- Define Architecture Decision Record (ADR) template and coding standards.

Deliverables

- Bootstrapped repository
- CI pipeline
- Local development instructions
- Environment config
- Baseline security scanning

Exit criteria

- A new engineer/agent can clone, configure and run the system consistently
- CI blocks obvious quality failures

Phase 3 - Conversation UX Prototype

Objective: Build the visible Pixel experience independently of the final AI pipeline.

Work to complete

- Design idle, listening, processing, speaking and error states.
- Implement microphone permission flow.
- Implement text input and transcript panel.
- Implement stop/mute/cancel controls.
- Create source card and recommended action components.
- Implement keyboard navigation, focus behavior and accessible labels.
- Use mocked responses so UX can be tested before the backend is complete.

Deliverables

- Interactive web prototype
- State machine for assistant UI
- Accessible input controls
- Mock conversation flows

Exit criteria

- Core conversation is understandable without instructions
- All voice actions have visual/text alternatives

Phase 4 - End-to-End Voice Loop

Objective: Prove the smallest real voice conversation from microphone to spoken answer.

Work to complete

- Implement audio capture and streaming/buffering.
- Integrate voice activity detection or explicit push-to-talk boundaries.
- Integrate speech-to-text or realtime speech model through an adapter.
- Send transcript to a minimal orchestrator.
- Generate a simple safe response.
- Integrate text-to-speech through an adapter.
- Play streamed or complete audio in the client.
- Implement barge-in: stop current speech, close/cancel generation and start a new turn.
- Measure latency for each stage.

Deliverables

- Working voice conversation
- Provider interfaces
- Latency instrumentation
- Interruption behavior

Exit criteria

- A user can complete multiple consecutive voice turns
- Barge-in works reliably
- Provider credentials never reach the browser except short-lived delegated session material where explicitly designed

Phase 5 - AI Orchestrator and Conversation State

Objective: Centralize decisions about intent, context, retrieval, tools and answer style.

Work to complete

- Define typed Turn, Message, Source, ToolCall and AssistantResponse models.
- Implement short-term conversation state.
- Implement intent routing with a small set of deterministic categories.
- Add system/developer policy loading and versioning.
- Add provider timeout/retry logic and cancellation.

- Add structured model outputs for tool/retrieval decisions where useful.
- Add output validation and safe fallback messages.

Deliverables

- Orchestrator module
- Conversation state store
- Intent routing
- Prompt/policy versioning
- Fallback behavior

Exit criteria

- All AI requests flow through one controlled orchestration boundary
- Conversation context is bounded and testable

Phase 6 - Cyber Florida Knowledge Ingestion and RAG

Objective: Make organization-specific answers grounded in approved current content.

Work to complete

- Create approved source registry.
- Build HTML/document importers.
- Normalize text while preserving headings, URLs and metadata.
- Chunk content using semantic boundaries rather than fixed character count alone.
- Generate embeddings and store chunks in PostgreSQL/pgvector or equivalent.
- Implement retrieval with metadata filtering.
- Add reranking if measurements show value.
- Return source IDs/URLs with every retrieved result.
- Create refresh/delete behavior for changed sources.
- Create an evaluation set of real Cyber Florida questions with expected sources.

Deliverables

- Ingestion pipeline
- Vector index
- Source registry
- Retrieval API
- Knowledge evaluation dataset

Exit criteria

- Answers cite correct sources on the evaluation set
- Deleted/outdated source content can be removed cleanly
- Retrieved content cannot directly invoke tools or override assistant policy

Phase 7 - Tools, Actions and Program Navigation

Objective: Move from answering questions to helping users complete safe next steps.

Work to complete

- Start with safe tools: `navigate_to_url`, `find_program`, `find_resource` and `search_approved_content`.
- Define strict JSON/schema contracts for every tool.
- Validate tool arguments server-side.
- Allowlist domains/routes for navigation tools.
- Add confirmation requirements to side-effecting actions.
- Record tool execution and result in audit/telemetry data.
- Never expose internal service credentials or raw privileged responses to the model.

Deliverables

- Tool registry
- Tool policy engine
- Navigation actions
- Tool tests
- Audit records

Exit criteria

- No arbitrary URL/tool execution
- Unauthorized users cannot access privileged tools
- Tool failure does not break the conversation session

Phase 8 - Security, Privacy and Abuse Resistance

Objective: Harden Pixel before broad user testing.

Work to complete

- Threat-model microphone, client, API, model, RAG, admin, tools and logging paths.
- Test prompt injection through user input and retrieved documents.
- Add server-side authorization and least-privilege service identities.
- Add secret management and key rotation procedure.
- Add rate limits, request limits and abuse controls.
- Implement log redaction and retention rules.
- Add security headers, CORS policy, dependency scanning and input validation.
- Create incident runbooks and kill switches for tools/providers.

Deliverables

- Threat model
- Security test suite
- Privacy/retention configuration
- Incident runbooks
- Abuse controls

Exit criteria

- Critical security findings are fixed
- Privileged actions are demonstrably protected
- Sensitive values are not present in standard logs

Phase 9 - Product Quality and Frontend Polish

Objective: Turn the engineering prototype into a usable Cyber Florida product.

Work to complete

- Apply Cyber Florida-approved visual identity and content tone.
- Tune listening/speaking animations so they represent real state rather than decorative motion.
- Add responsive mobile/tablet/desktop behavior.
- Improve source cards, action buttons, error states and empty states.
- Tune transcript presentation and history limits.
- Conduct accessibility review and user testing with keyboard/screen reader where possible.
- Optimize audio buffering and prevent duplicate playback.

Deliverables

- Production UI
- Responsive states
- Accessibility fixes
- UX issue log resolved

Exit criteria

- Users can understand Pixel state at all times
- No core flow depends only on color, sound or pointer interaction

Phase 10 - Observability and Operations

Objective: Make production behavior measurable and supportable.

Work to complete

- Define structured events for session start, turn start/end, STT, retrieval, model, TTS, tool and errors.
- Add correlation IDs to every turn.
- Build dashboards for latency, error rate, volume and provider failures.
- Add knowledge quality metrics such as no-result rate and low-score retrieval rate.
- Create alerts for elevated failure rates and exhausted quotas.
- Create admin views or reports for feedback and knowledge ingestion failures.

Deliverables

- Dashboards
- Alerts
- Logging/tracing
- Operational runbook
- Feedback review flow

Exit criteria

- A failed user turn can be diagnosed from telemetry without reading sensitive raw content
- Core dependency failures trigger alerts

Phase 11 - Comprehensive Testing and Evaluation

Objective: Prove Pixel works correctly, safely and consistently before release.

Work to complete

- Run unit tests for policies, routing, chunking, tool validation and transformations.
- Run integration tests with provider sandboxes/mocks.
- Run end-to-end voice tests in supported browsers/devices.
- Run RAG evaluation against expected sources and answer criteria.
- Run prompt injection and cyber-safety red-team cases.
- Run accessibility testing.
- Run load and soak tests for expected concurrency.
- Run failure injection: STT down, model timeout, no retrieval result, TTS failure, dropped network.
- Conduct stakeholder user acceptance testing.

Deliverables

- Test report
- Evaluation scores
- Defect list
- Security findings
- UAT sign-off criteria

Exit criteria

- No unresolved release-blocking defects
- Quality/safety thresholds are met
- Fallbacks operate correctly during dependency failure

Phase 12 - Staging Pilot

Objective: Release to a controlled group and learn from real conversations without full public exposure.

Work to complete

- Deploy production-like staging configuration.
- Seed final approved knowledge sources.
- Invite a limited set of testers representing target user groups.
- Collect feedback on accuracy, discoverability, tone, latency and voice quality.
- Review misunderstood intents and no-result retrievals.
- Fix high-frequency failure modes and update evaluation datasets.

Deliverables

- Pilot build
- Pilot feedback report
- Updated eval dataset
- Resolved priority issues

Exit criteria

- Stakeholders approve production launch
- Known limitations are documented and acceptable

Phase 13 - Production Release

Objective: Launch Pixel with controlled rollout, rollback capability and operational ownership.

Work to complete

- Create production environment and production secrets independently from staging.
- Run database/index migrations through versioned deployment steps.
- Run smoke tests immediately after deployment.
- Use canary/limited traffic rollout when supported.
- Verify dashboards, alerts, backups and rollback procedure.
- Publish user-facing privacy/AI notices and support path as required.
- Record release version for frontend, backend, prompt/policy and knowledge index.

Deliverables

- Production release
- Release notes
- Rollback artifact
- Support ownership
- Monitoring active

Exit criteria

- Smoke tests pass
- Operational owners confirm monitoring
- Rollback is tested or verified

Phase 14 - Continuous Improvement

Objective: Improve Pixel using measured failures, verified content changes and controlled experiments.

Work to complete

- Review feedback and failed intents regularly.
- Refresh and validate Cyber Florida content.
- Add regression tests for every major production failure.
- Measure whether prompt/model/retrieval changes improve evaluation scores before deployment.

- Retire unused tools and stale sources.
- Review privacy retention and security findings periodically.
- Plan new features only after the core quality metrics remain stable.

Deliverables

- Updated knowledge
- Regression tests
- Quality trend reports
- Controlled roadmap improvements

Exit criteria

- Every material production issue results in a test, control or documented decision
- Knowledge freshness and safety remain measurable

07

5. Step-by-Step Engineering Build Plan

The sequence below is intentionally ordered so each group of steps produces a working increment. An AI coding agent can execute these as epics/tasks while preserving the architecture and release gates above.

STEP	IMPLEMENTATION TASK
1	Create /docs/product.md with Pixel vision, target users, MVP, out-of-scope and success criteria.
2	Create /docs/policies.md with identity, voice response rules, uncertainty language, cyber-safety boundaries and escalation rules.
3	Create /docs/architecture.md and record the first ADRs: frontend framework, backend framework, database/vector strategy, real-time transport and provider abstraction.
4	Create repository structure and root README with local development commands.
5	Configure frontend TypeScript, linting, formatting, tests and environment handling.
6	Configure FastAPI backend with typed settings, health endpoint, linting, formatting and tests.
7	Add Dockerfiles/compose or equivalent local environment; include Postgres/pgvector and optional Redis.
8	Create CI pipeline for lint, type check, unit tests, build and dependency scan.
9	Create frontend assistant state machine: idle -> listening -> processing -> speaking -> error.
10	Build microphone permission and input selection UI.
11	Build text input so development never depends on working audio.
12	Build transcript/history component with clear user vs. Pixel turns.
13	Build stop/mute/clear controls and keyboard behavior.
14	Create a mock conversation provider for deterministic UI development.
15	Define VoiceProvider interface: connect, startTurn, sendAudio, endTurn, cancel, onTranscript, onAudio, onError.
16	Implement the first STT/realtime adapter behind VoiceProvider.
17	Define TTSPProvider interface if TTS is separate; implement first adapter.
18	Implement audio playback queue and cancellation.
19	Implement barge-in logic that cancels model/TTS/audio before starting the new turn.
20	Instrument audio/STT/TTS latency metrics.

STEP	IMPLEMENTATION TASK
21	Define shared models: Session, Turn, Message, SourceRef, Citation, ToolCall, ToolResult, AssistantResponse.
22	Create POST /v1/sessions and text POST /v1/messages.
23	Implement bounded session conversation context.
24	Create versioned Pixel system policy and load it server-side.
25	Create intent enum and router: cyberflorida_knowledge, cybersecurity_help, scam_help, navigation, clarification, unsupported.
26	Add provider timeout, retry and cancellation behavior.
27	Create a safe generic fallback when AI/provider calls fail.
28	Create database migrations for source registry, documents, chunks, embeddings metadata, feedback and audit events.
29	Build source registry with allowlisted Cyber Florida URLs/documents.
30	Build HTML importer that extracts main content and preserves title/headings/canonical URL/update metadata.
31	Build document importer for approved PDF/DOCX/text sources as required.
32	Normalize content and remove navigation/boilerplate without deleting meaning.
33	Implement semantic chunking and stable chunk IDs.
34	Generate embeddings and write chunks to pgvector.
35	Implement retrieval with top-k search plus metadata filters.
36	Add reranking only after establishing a retrieval baseline.
37	Return source metadata with retrieval results.
38	Update orchestrator so Cyber Florida questions require retrieval before answer generation.
39	Add grounded-answer rule: if evidence is missing/weak, Pixel must say it cannot verify the answer.
40	Build source cards in UI and link answer/source associations.
41	Create 100+ knowledge evaluation questions spanning organization overview, programs, audiences, contacts/resources and current-information cases.
42	Score retrieval hit rate and grounded answer correctness; fix chunking/query/retrieval before changing prompts randomly.
43	Define Tool interface with name, description, input schema, required permission, confirmation policy and execute method.
44	Implement navigate_to_url with strict route/domain allowlist.
45	Implement find_program/find_resource using structured approved data or controlled retrieval.
46	Add server-side tool argument validation and tool timeout handling.
47	Add audit events for tool calls and administrative source changes.
48	Implement prompt injection defenses: retrieved content cannot create tool permissions or modify system policy.
49	Implement rate limiting, request/body limits, CORS/security headers and input normalization.
50	Move production secrets to a managed secrets store and document rotation.
51	Implement log redaction and content retention configuration.
52	Create dashboards for turn volume, p50/p95 latency by stage, model errors, STT/TTS errors, retrieval no-result rate and tool failures.
53	Create feedback endpoint and feedback UI.
54	Create admin ingestion/reindex workflow protected by authentication/authorization.

STEP	IMPLEMENTATION TASK
55	Write unit tests for routing, policy decisions, tool schemas, chunking, source authorization and failure fallbacks.
56	Write integration tests for message -> retrieval -> model -> response with provider mocks.
57	Write E2E browser tests for text mode and voice state transitions.
58	Create safety/red-team test set including prompt injection, data exfiltration attempts, tool abuse and harmful cyber requests.
59	Create performance tests for concurrent sessions and audio/message throughput.
60	Run accessibility testing and remediate keyboard, label, contrast, focus and transcript issues.
61	Deploy staging with production-like networking, database and secrets patterns.
62	Run UAT/pilot; log every failure as product, retrieval, model, voice, tool, UI or content issue.
63	Add regression test/eval case for each release-blocking pilot failure.
64	Create production deployment, rollback, incident, secret rotation and knowledge refresh runbooks.
65	Release gradually, monitor quality/latency/errors, and stop/roll back if release gates are violated.

5.1 First vertical slice to build

BUILD THIS BEFORE EVERYTHING ELSE

A web page with a microphone button -> user speaks -> transcript appears -> backend receives text -> Pixel returns one short answer -> answer is spoken -> user interrupts -> the previous audio stops and the new turn begins. Once this works, replace the generic answer with Cyber Florida RAG. This validates the hardest end-to-end interaction before the project grows.

5.2 Recommended data model (minimum)

ENTITY	KEY FIELDS / PURPOSE
sessions	id, user/auth context reference, created_at, expires_at, configuration flags
turns	id, session_id, role, input_mode, status, timing fields, created_at
sources	id, canonical_url/path, title, source_type, access_class, active, last_fetched_at
documents	id, source_id, content_hash, version, extracted metadata, status
chunks	id, document_id, ordinal, heading_path, content, token_count, embedding, metadata
feedback	id, turn_id, rating/category, optional comment, created_at
audit_events	id, actor/context, action, target, result, timestamp, trace/correlation id

5.3 Environment strategy

ENVIRONMENT	RULES
Local	Mock providers by default; optional real provider keys; local Postgres/pgvector; synthetic test data.

ENVIRONMENT	RULES
Development	Shared integration environment; non-production keys and sources; frequent deploys.
Staging	Production-like configuration, auth and monitoring; approved content copy; pilot/UAT.
Production	Separate credentials/data, strict admin access, monitoring/alerts, controlled releases, backups and rollback.

08

6. Testing and Evaluation Strategy

Traditional software tests are necessary but not sufficient for an AI voice assistant. Pixel needs deterministic tests for code plus evaluation datasets for retrieval, answers, conversation behavior and safety.

6.1 Test layers

TEST TYPE	WHAT TO VERIFY
Unit	State transitions, routing, chunking, metadata filters, tool validation, authorization policies, redaction, fallback logic.
Contract	Provider adapters, tool schemas, API request/response models, database migrations.
Integration	Message -> orchestrator -> retrieval -> AI -> response; ingestion -> index -> search; tool selection -> validation -> execution.
End-to-end	Browser microphone/text flow, transcript, speaking, interruption, sources, clear session, error recovery.
Voice quality	Recognition of cybersecurity terminology, background noise, accents, silence, fast speech, interruption, audio device changes.
RAG evaluation	Correct source retrieved, answer supported by source, no unsupported organization claims, correct freshness behavior.
Safety/red-team	Prompt injection, system prompt extraction, secret requests, tool abuse, malicious retrieved text, harmful cyber requests.
Performance	Concurrent sessions, first response latency, sustained sessions, database/vector query latency, provider throttling.
Accessibility	Keyboard-only, screen-reader semantics, focus order, captions/transcript, reduced-motion behavior where applicable.
UAT	Real target users complete representative tasks and report trust, usefulness, clarity and friction.

6.2 RAG evaluation metrics

- Retrieval hit rate: expected authoritative source appears in top results.
- Context precision: retrieved passages are relevant rather than merely keyword-related.
- Groundedness: important factual claims in the answer are supported by retrieved evidence.
- Answer correctness: answer matches expected facts for the test case.
- Citation/source correctness: displayed source actually supports the answer.
- Abstention quality: Pixel refuses to invent an answer when evidence is missing.
- Freshness: time-sensitive questions use current indexed data and do not rely on stale content.

6.3 Voice performance metrics

METRIC	PURPOSE
Speech start detection	Time from user speech to recognized active turn.

METRIC	PURPOSE
End-of-turn detection	Delay between user finishing and assistant processing.
Transcript accuracy	Manual sample review plus domain-word error analysis.
Time to first response audio	Time from end of user turn to first audible Pixel response.
Barge-in cancellation	Time required to stop current Pixel audio after new user speech/interrupt.
Conversation completion	Percentage of representative tasks completed without switching to manual workaround.

6.4 Required failure tests

- Microphone permission denied.
- No microphone found / microphone disconnected.
- User says nothing after entering listening mode.
- Network drops mid-turn.
- Speech provider returns an error or malformed transcript.
- LLM times out or is rate-limited.
- Retriever returns zero acceptable sources.
- Indexed source was removed or marked inactive.
- TTS fails after a valid text response.
- Tool returns an error or unauthorized result.
- User interrupts repeatedly while Pixel is speaking.
- Malicious instructions are embedded in retrieved content.
- User asks Pixel to reveal prompts, credentials or hidden internal information.

09

7. Deployment, CI/CD and Operations

7.1 CI/CD release pipeline

1. Developer/agent opens change with linked requirement or issue.
2. CI runs formatting, linting, type checks, unit tests, build and dependency scanning.
3. AI/RAG changes also run the relevant evaluation suite and compare results with the current baseline.
4. Change is deployed to development and then staging.
5. Staging smoke/E2E tests run against production-like dependencies.
6. Approved changes are promoted to production using a versioned artifact, not rebuilt from an unreviewed working tree.
7. Post-deploy smoke tests verify health, database/index compatibility, conversation path and source retrieval.
8. Monitoring is watched for error/latency regressions; rollback path remains immediately available.

7.2 Version the AI system, not only the code

- Application frontend version.
- Backend/orchestrator version.
- System policy/prompt version.
- Model/provider configuration version.
- Knowledge index snapshot/version.
- Tool registry/policy version.
- Evaluation dataset version.

WHY THIS MATTERS

When an answer changes, the team must be able to determine whether the cause was code, prompt/policy, AI model, retrieval data, a tool, or source content. Treat these components as versioned production artifacts.

7.3 Operational dashboards

DASHBOARD	SIGNALS
Availability	Successful session creation, turn success rate, dependency health.
Latency	p50/p95 stage timings for STT, retrieval, model, TTS and total turn.
Knowledge	No-result queries, low-relevance queries, most-used sources, ingestion failures.
Tools	Calls by tool, failures, denied calls, confirmation/cancellation rates.
Quality	User feedback, repeated questions, escalations, common misunderstood intents.
Safety/security	Rate-limit events, suspicious tool attempts, auth failures, injection detections, admin changes.

7.4 Knowledge maintenance process

1. Content owner approves authoritative sources.
2. Ingestion job fetches/imports content and calculates a content hash.
3. Changed content is normalized, chunked and embedded.
4. Automated sanity checks verify minimum content, source metadata and no parser failures.
5. A small knowledge regression suite runs against the updated index.
6. If checks pass, the index/source version is promoted; otherwise the previous version remains active.
7. Removed/outdated sources are disabled and their chunks are no longer retrieved.

7.5 Incident response minimums

- Disable or revoke a compromised provider credential.
- Disable a specific tool without redeploying the whole application where practical.
- Deactivate a bad/stale knowledge source and re-index.
- Roll back application and policy versions.
- Identify affected sessions/turns using correlation IDs.
- Preserve necessary audit data without expanding collection beyond policy.
- Communicate user-visible limitations or outages through an approved channel.

10

8. Major Risks and Controls

RISK	SEVERITY	CONTROL
Hallucinated Cyber Florida facts	High	Require RAG for organization-specific facts; show sources; abstain when evidence is insufficient; maintain evaluation set.
Stale deadlines/events	High	Track content freshness; refresh authoritative sources; treat date-sensitive intents specially.
Prompt injection from webpages/docs	High	Treat retrieval as untrusted data; fixed system policy; tool allowlists; separate authorization layer.
Voice latency feels slow	High	Streaming, VAD/end-turn tuning, stage-level telemetry, shorter voice answers, caching where appropriate.
Speech recognition mistakes	Medium	Text transcript visible, edit/retry path, domain term hints where supported, measure error cases.
Sensitive information in logs	High	Data minimization, redaction, structured

RISK	SEVERITY	CONTROL
		telemetry, retention limits, access controls.
Unauthorized tool action	High	Server-side authz, schema validation, confirmation policy, allowlists, audit trail.
Vendor lock-in	Medium	Provider interfaces and normalized internal data models; test alternate providers where business-critical.
Overly broad MVP	High	Lock MVP use cases and release gates; defer multimodal/personalization/deep integrations until core quality is stable.
User over-trust	Medium	AI disclosure, visible sources, calibrated uncertainty, no false claims of certainty.
Knowledge ingestion errors	Medium	Content hashes, parser tests, source metadata, preview/QA, versioned index and rollback.
Accessibility gaps	Medium	Build text alternative from day one; keyboard and screen-reader testing before production.

8.1 Features that should NOT be built first

- Long-term personal memory.
- Autonomous execution of security actions.
- Large number of third-party integrations.
- Complex avatars/3D characters before voice responsiveness is proven.
- Multiple mobile platforms before the web architecture and product behavior stabilize.
- Broad internet browsing without source governance.
- A huge vector database before retrieval quality is measured on the actual Cyber Florida corpus.

11

9. Production Definition of Done

Pixel should not be considered production-ready because a demo works. The following conditions form a practical release checklist.

Product

- [] MVP scope is implemented and deferred features are documented.
- [] Top user journeys have owners and acceptance tests.
- [] Pixel identity, disclosure and limitations are approved.

Voice

- [] Listen/transcribe/respond/speak works on supported platforms.
- [] Barge-in and stop controls work.
- [] Voice errors degrade to usable text interaction.

Knowledge

- [] Approved Cyber Florida sources are indexed and traceable.
- [] Evaluation set meets approved quality threshold.
- [] Pixel abstains when authoritative evidence is insufficient.
- [] Content update/deletion process is verified.

Security & privacy

- [] Threat model reviewed.
- [] Critical/high findings resolved or formally accepted.
- [] Secrets are server-side and managed.

- ☐ Authorization is enforced server-side.
- ☐ Logging/retention behavior matches approved policy.
- ☐ Prompt injection/tool abuse tests pass at the release threshold.

Accessibility

- ☐ Core flows work by keyboard.
- ☐ Inputs and states have accessible labels.
- ☐ Transcript/text alternatives exist.
- ☐ Manual and automated checks have no unresolved release-blocking issues.

Operations

- ☐ Staging and production are separated.
- ☐ Dashboards and alerts work.
- ☐ Rollback procedure is documented and verified.
- ☐ Knowledge administration and feedback review have owners.
- ☐ Incident runbooks exist.

Quality

- ☐ Unit/integration/E2E tests pass.
- ☐ RAG/safety evals pass.
- ☐ Pilot/UAT release blockers resolved.
- ☐ Known limitations are documented.

FINAL PRINCIPLE

Pixel is ready when it is trustworthy under normal use and predictable under failure - not merely when it can answer impressive questions in a controlled demo.

12

10. Instructions for an AI Coding Agent

If an AI development agent is used to build Pixel, give it the project description plus the rules below. These rules prevent the agent from optimizing for a demo at the expense of security, maintainability or testability.

- Work phase-by-phase. Do not skip foundation, security or evaluation steps to add cosmetic features.
- Before modifying architecture, read `/docs/product.md`, `/docs/policies.md` and `/docs/architecture.md`.
- Keep provider-specific SDK code inside adapters. Core orchestration must use internal interfaces/types.
- Never put provider API secrets in frontend code or commit secrets to the repository.
- Every new tool must have a typed schema, server-side validation, permission policy, error handling and tests.
- Every organization-specific factual answer must use the knowledge layer unless an explicitly curated deterministic facts source covers it.
- Every production bug in routing, knowledge, tools, security or safety should produce a regression test/evaluation case.
- Do not add a dependency without explaining why it is needed and checking licensing/security implications.
- Do not silently change data retention, authentication, authorization or logging behavior.
- Before declaring a phase complete, run tests, build the application and verify the phase exit criteria.
- Prefer simple measurable implementations over premature distributed/microservice complexity.
- Document known limitations and decisions rather than hiding them behind prompts.

Suggested implementation order for the agent

1. Foundation and mocked UI
2. Real voice loop
3. Orchestrator
4. Cyber Florida RAG

- 5. Sources in UI
- 6. Safe navigation tools
- 7. Security/privacy hardening
- 8. Evaluation suite
- 9. Observability/admin workflow
- 10. Staging pilot
- 11. Production release

13

11. Source Notes and Recommended Standards

Cyber Florida context used in this document is based on official Cyber Florida at USF information. Technical architecture and development recommendations are design guidance for this project and should be validated against Cyber Florida/USF technology, security, procurement, accessibility and privacy requirements before production adoption.

REFERENCE	WHY IT MATTERS	URL
[1] Cyber Florida at USF - Home	Cyber Florida describes itself as a state-funded organization working to position Florida as a national leader in cybersecurity through education, research and outreach.	https://cyberflorida.org/
[2] Cyber Florida at USF - Mission & History	Official background on The Florida Center for Cybersecurity / Cyber Florida and its establishment within the University of South Florida.	https://cyberflorida.org/about/
[3] NIST AI Risk Management Framework	Recommended reference for governing, mapping, measuring and managing AI risk.	https://www.nist.gov/itl/ai-risk-management-framework
[4] OWASP Top 10 for Large Language Model Applications	Recommended reference for LLM application security risks, including prompt injection and related controls.	https://owasp.org/www-project-top-10-for-large-language-model-applications/
[5] W3C Web Content Accessibility Guidelines (WCAG) 2.2	Recommended reference for web accessibility requirements and testing.	https://www.w3.org/TR/WCAG22/

Closing project statement

Pixel should be built as a product platform, not as a single prompt. The durable value comes from the combination of a strong voice experience, a governed Cyber Florida knowledge layer, safe AI orchestration, constrained tools, clear security/privacy boundaries, measurable quality, and an operational process that keeps answers accurate after launch.